الملف التاسع: الفهرس الشامل والمفصّل لجميع ملفات النظام (File-by-File Breakdown)

بناءً على طلبك بشرح **كل ملف وكل جزء في المشروع بلا استثناء**، قمنا في هذا الدليل بالدخول إلى أصغر وحدة بناء في النظام (الملفات البرمجية ذاتها) وشرح وظيفة كل ملف على حدة وكيف يعمل ضمن المكون الخاص به.

1. وكيل نظام ويندوز (win_edrAgent)

الوكيل هو المسؤول عن جمع الأحداث والاستجابة. يحتوي على الملفات التالية:

مجلد `/cmd/agent` (نقطة البداية)

- `main.go` : نقطة الانطلاق الرئيسية (Entrypoint). يقوم بقراءة الإعدادات، التحقق من تسجيل الجهاز (Enrollment)، ثم يبدأ جميع الـ Goroutines الخاصة بجمع البيانات، ويبدأ الاتصال بـ gRPC.
- `service_windows.go` : يغلف الوكيل ليعمل كـ "Windows Service" في الخلفية حتى بدون تسجيل دخول مستخدم.

مجلد `/internal/collectors` (جامعات البيانات)

- `etw_cgo.c` / `etw_cgo.h` : كود لغة C منخفض المستوى للتعامل مع Windows API. يفتح جلسات الـ ETW ويتلقى الأحداث من النواة.
- `etw.go` : يستدعي كود C وينقل الأحداث إلى بيئة Go لمعالجتها وإثرائها (Enrichment).
- `wmi.go` : يتصل بـ WMI عبر COM لجمع معلومات النظام الثابتة.
- `network.go` : يراقب اتصالات الشبكة الحية والمنافذ المفتوحة.
- `registry.go` : يراقب تعديلات السجل (Registry) للبحث عن آليات الثبات (Persistence).
- `file.go` / `process.go` / `imageload.go` : تقوم بتصنيف الأحداث الواردة من ETW وتغليفها بهياكل بيانات جاهزة للإرسال.
- `usb_watcher.go` : يراقب إدخال وإخراج أجهزة التخزين (Flash Drives) عبر استعلامات WMI.
- `vuln_scanner.go` : يفحص البرامج المثبتة ويقاطعها مع قواعد بيانات CVE محلياً.
- `filter.go` / `rate_limiter.go` : يفلتر الأحداث المزعجة (Noise) ويحدد سقفاً لعدد الأحداث في الثانية لمنع استهلاك المعالج.

مجلد `/internal/responder` (الاستجابة التلقائية)

- `engine.go` : المحرك الأساسي الذي يقيم الأحداث فور وقوعها للتدخل.

- `process_engine.go` : يراقب سلوك العمليات ويقتلها إن خالفت قواعد معينة.
- `quarantine.go` / `quarantine_restore.go` : ينقل الملفات الخبيثة إلى مجلد معزول، ويشفرها، ويسمح باستعادتها عند طلب لوحة التحكم.

مجلد `/internal/command` (معالجة أوامر C2)

- `handler.go` : يستقبل الأوامر من خادم مدير الاتصالات ويوجهها.
- `executor.go` : ينفذ أوامر الطرفية (Shell) والأوامر البرمجية المباشرة.
- `sysmon.go` : يدير تثبيت وتفعيل وتعطيل خدمة Sysmon لجمع سجلات إضافية.
- `audit.go` / `forensics_test.go` : يجمع سجلات التحقيق الجنائي والأحداث من Event Viewer (تُستخدم لأمر `collect_logs`).

مجلد `/internal/grpc` و `/pb` (الشبكات وبروتوكول الاتصال)

- `client.go` : يفتح قناة اتصال طويلة الأمد عبر gRPC مع الخادم.
- `mtls.go` : يحمل الشهادات الموقعة ويفعل المصادقة الثنائية.
- `heartbeat.go` : يرسل إشارة "أنا حي" كل 30 ثانية مع مقاييس استهلاك المعالج والذاكرة.
- `reconnect.go` : يتولى محاولة إعادة الاتصال المتزايدة (Exponential Backoff) عند انقطاع الشبكة.
- `edr.proto` / `edr.pb.go` : ملفات تعريف هياكل البيانات المتفق عليها (Protobuf) بين الوكيل والخادم لضمان صغر حجم البيانات المُرسلة.

مجلدات الحماية والنظام (Queue, Security, Config, Enrollment)

- `queue/disk_queue.go` : يكتب الأحداث على القرص الصلب (WAL) عندما ينقطع الإنترنت لتجنب ضياعها.
- `enrollment/enroll.go` : يسجل الجهاز لأول مرة ويجلب له شهادة رقمية (Certificate) باستخدام الرمز السري المؤقت.
- `security/acl_windows.go` : يعبث بصلاحيات نظام الملفات (NTFS) ليمنع حتى مدراء النظام (Admins) من مسح أو تعديل ملفات الوكيل (Anti-Tamper).
- `config/config.go` : يقرأ الإعدادات المحلية المشفرة من الـ Registry.

2. مدير الاتصالات (connection-manager)

مدير الاتصالات هو العصب المركزي، يحتوي على الملفات التالية:

مجلد `/cmd/server`

- `main.go` : يربط قاعدة البيانات، Redis، وخادم gRPC وخادم REST API ويبدأ تشغيلهم جميعاً.

مجلد `/internal/handlers` (واجهات التخاطب)

- `grpc_server.go` : يستقبل الدفق (Stream) من الوكلاء، يتحقق من هوية كل وكيل، ويمرر الأحداث إلى Kafka.
- `agent_handlers.go` : يوفر الـ API لصفحة الأجهزة في لوحة التحكم (عرض الوكلاء، تفاصيل وكيل).
- `command_handlers.go` : يستقبل طلب إصدار أمر من لوحة التحكم ويرسله لـ gRPC.
- `auth_handlers.go` : يوفر API لتسجيل دخول محققي الأمن وإصدار الـ JWT.
- `automation_handlers.go` : يوفر API لإدارة قواعد الأتمتة (Playbooks).
- `vulnerability_handlers.go` : يوفر API لعرض الثغرات المكتشفة بالأنظمة.

مجلد `/internal/service` (عقل العمليات - Business Logic)

- `agent_service.go` : يتتبع صحة وحالة جميع الأجهزة ويقيم الـ Health Score.
- `command_service.go` : يربط أوامر لوحة التحكم بالوكلاء المعنيين عبر قنوات الـ gRPC النشطة.
- `auth_service.go` : يدير دورة حياة الجلسات، التشفير بـ bcrypt، وحظر المستخدمين المتلاعبين.
- `automation_service.go` : العقل الذي يربط تنبيهاً جديداً بـ Playbook مناسب، وينفذ سلسلة من الأوامر تلقائياً.
- `certificate_service.go` : يدير الـ PKI، يوقع شهادات الوكلاء (mTLS) ويراقب انتهاء صلاحيتها.
- `ml_optimizer.go` : يستخدم خوارزميات إحصائية لتحسين معدلات نجاح قواعد الأتمتة.
- `kev_sync_service.go` : يتصل بموقع وكالة CISA لجلب قائمة الثغرات المستغلة حديثاً (KEV) ومقاطعتها مع الأنظمة.

مجلد `/internal/repository` (قواعد البيانات)

- `agent_repo.go` , `command_repo.go` , `alert_repo.go` : كل ملف منها يحتوي على استعلامات SQL النظيفة والمؤمنة ضد الحقن للتحدث مع PostgreSQL.

مجلد `/internal/cache` (التخزين المؤقت)

- `redis.go` : يدير اتصال Redis.
- `pubsub.go` : ينشر (Publishes) إشعارات عن أي تنبيه جديد ليصل للوحة التحكم فوراً عبر WebSocket.

3. محرك الكشف سيجما (sigma_engine_go)

يعمل كـ Stream Processor. كل ملفاته متخصصة في التحليل المعقد:

مجلد `/cmd/sigma-engine`

- `main.go` : يشغل مستهلك (Consumer) Kafka وخادم REST API الخاص بـ Sigma.

مجلد `/internal/application/rules`

- `rule_parser.go` : يأخذ ملف الـ YAML، يحلل تركيبه اللغوي، ويبني شجرة شروط (Abstract Syntax Tree).
- `indexer.go` : ينشئ فهرساً في الذاكرة لتسريع البحث عن القاعدة المطابقة دون المرور على آلاف القواعد (O(1) Lookup).

مجلد `/internal/domain` (منطق الكشف)

- `evaluator.go` : ينفذ معاملات المقارنة (هل النص يحتوي كذا، هل يطابق Regex كذا).
- `aggregation.go` : يمنع إغراق المحلل بالتنبيهات. يجمع 10 أحداث مشبوهة متشابهة في تنبيه واحد.
- `confidence_scorer.go` : يحسب "درجة الثقة" للكشف. الكشوفات الضعيفة جداً يتم فلترتها كـ False Positives.
- `alert_generator.go` : يأخذ الحدث المتطابق والقاعدة وينتج كيان "تنبيه" (Alert) متكامل.

مجلد `/internal/analytics` و `/ml`

- `correlation_manager.go` : يربط بين أحداث متعددة وقعت في أوقات مختلفة لكنها جزء من هجوم واحد (مثلاً: عملية نزلت ملفاً، والملف اتصل بالإنترنت).
- `risk_scorer.go` : يحسب "نقاط الخطر" لجهاز معين بناءً على أهميته وخطورة التنبيهات التي أصابته.
- `ueba.go`: يكتشف الانحرافات في سلوك المستخدمين (مثلاً: مستخدم عادي فتح أداة إدارية غير معتادة) (User and Entity Behavior Analytics).

مجلد `/internal/handlers`

- يوفر الـ API لصفحات إدارة التنبيهات وقواعد Sigma في لوحة التحكم (إضافة/حذف قواعد، وتحديث حالات التنبيهات).

4. لوحة التحكم (Dashboard)

لوحة التحكم تتكون من المجلدات والملفات التالية:

مجلد `/src/api` (التواصل مع الخوادم)

- `client.ts` : يحتوي على `axios` Interceptors لحقن توكن الـ JWT، وهو الملف الوحيد الذي يعلم بوجود الـ Backend.
- يحتوي على دوال (Wrappers) جاهزة مثل `alertsApi.get` ، `agentsApi.list()` تستخدمها واجهات العرض.

مجلد `/src/pages` (الصفحات المرئية)

- `Dashboard.tsx` : واجهة الـ NOC تعرض إحصائيات عامة والـ Timeline للتهديدات.

- `Endpoints.tsx` : صفحة عرض جدول الأجهزة.
- `EndpointDetail.tsx` : أكبر ملف؛ يعرض كل شيء يخص جهاز واحد (عمليات حية، سجلات، تنبيهات، وأزرار إصدار الأوامر كالعزل والإيقاف).
- `Alerts.tsx` : صفحة الـ Inbox للمحللين لفرز التنبيهات.
- `VulnerabilityManagement.tsx` : صفحة تعرض الثغرات وترتبها بأولوية الخطر (KEV vs CVSS).
- `Rules.tsx` : واجهة إدارة (تعديل/تفعيل) قواعد سيجما.
- `ActionCenter.tsx` : شاشة لمراقبة تقدم الأوامر المرسلة للوكلاء لحظة بلحظة.
- `AuditLogs.tsx` : صفحة التدقيق والمساءلة (من فعل ماذا).
- `automation/PlaybooksPage.tsx` و `AutomationRulesPage.tsx` : واجهات بناء شروط الأتمتة وخطوات رد الفعل التلقائي (Playbooks).
- `AgentDeployment.tsx` : صفحة توليد روابط تثبيت الوكلاء والرموز السرية للمنظمة.

مجلد `/src/components`

- `Modal.tsx` , `Drawer.tsx` , `Table.tsx` , `StatusBadge.tsx` : مكونات UI يعاد استخدامها في كل الصفحات للحفاظ على تناسق التصميم (Design System).

مجلد `/src/hooks`

- `useWebSocket.ts` : كود حي يتصل بالخادم ويحدث الشاشة في الوقت الفعلي عندما يقع تنبيه جديد.

5. خدمة بناء الوكلاء (Agent Builder)

- `main.go` / `api.go` : واجهة API تستقبل طلبات البناء.
- `builder.go` : المحرك الذي يأخذ الكود المصدري لوكيل الويندوز، يحقن فيه عنوان خادم ה- C2 ورمز التسجيل (Token)، ثم يشغل `go build` وينتج ملف `agent.exe` مخصص للمستخدم طالبه من لوحة التحكم.

6. البنية التحتية والسكربتات (Infrastructure & DB)

- `docker-compose.yml` : المخطط الشامل الذي يشغل قواعد البيانات (PostgreSQL)، التخزين المؤقت (Redis)، منصة التدفق (Kafka)، والـ Backends، ويضبط الشبكات الداخلية بينهم.
- `nginx/nginx.conf` : يعمل كـ Reverse Proxy يستقبل كل الطلبات من المتصفح ويوجهها داخلياً إلى Service المناسبة (لتخطي مشاكل CORS وتوحيد منافذ الدخول).
- `db/migrations/*.sql` : سلسلة ملفات SQL تحمل شكل جداول قاعدة البيانات (Schema) يتم تنفيذها بالترتيب عند تشغيل Connection Manager لضمان اتساق الجداول مع الكود البرمجي.

- `genproto.ps1` , `scripts/build.ps1` : سكربتات لتسهيل توليد كود الـ Protobuf للمطورين (تستخدم أثناء كتابة الكود لإنشاء واجهات الـ gRPC بين الوكيل ومدير الاتصالات).